

LISP Interpreter Report

Amairany Neri

New Mexico Institute of Mining and Technology Department of Computer Science
Socorro, NM, USA

amairany.neri@student.nmt.edu

ABSTRACT

In this paper, we discuss the implementation of an interactive LISP interpreter using Python. The main goal of this project was to understand the concepts behind creating an interpreter for the simple functional programming language LISP.

CCS CONCEPTS

• General and reference;

KEYWORDS

LISP, Interpreter, Functional Programming, Python

1 INTRODUCTION

The LISP Interpreter was created using Python. Very similar to the actual LISP interpreter, you input LISP expressions and the interpreter evaluates it on the line below. An output file of the session is generated under the name `results`.file.

2 LISP CONSTRUCTS IMPLEMENTED

Below is a table of the status of each LISP construct implemented in the interpreter.

Table 1: Project Progress/Final Status Reporting

Lisp Construct	Status
Variable reference	DONE
Constant literal	DONE
Quotation	DONE
Conditional	DONE
Variable Definitions	DONE
Functional call	DONE
Assignment	DONE
Function Definition	DONE
The arithmetic +, -, *, / operators on INTEGER type	DONE
"car" and "cdr"	DONE
The built-in function: "cons"	DONE
sqrt, exp	DONE
>, <, =, <=, >=, !=	DONE

¹ set! lacks error checking for formal parameters.

² Division uses floating-point arithmetic.

The following functions for extra credit were also implemented:

3 IMPLEMENTATION DESCRIPTIONS

The interpreter implements core Lisp constructs through a combination of recursive parsing and environment-based evaluation [2][1]. Variable references are resolved via the `Environment.find()` method, which recursively searches parent scopes until a match is found.

Extra credits	Status
"mapcar"	DONE
"lambda"	DONE

Table 2: Extra Credits Functions

Quotation is handled during parsing by wrapping quoted expressions in a `(quote ...)` list that bypasses evaluation. For conditionals, the `if` special form evaluates the test expression first, then selectively evaluates either the consequent or alternative branch using Python's ternary logic. Function calls leverage Python's first-class function capabilities. Built-in operators like `+` and `car` are stored as lambda functions in the global environment dictionary, while user-defined functions created via `defun` or `lambda` are instantiated as Procedure objects. These objects capture their defining environment during creation to enable static scoping, ensuring proper variable resolution during subsequent calls.

4 APPLIED CLASS CONCEPTS

The implementation directly applies several core concepts from programming language theory. Static scoping is achieved through environment chaining, where each Procedure maintains a reference to its birth environment. Recursive descent parsing techniques learned during syntax analysis modules were adapted to convert token streams into abstract syntax trees. The interpreter's memory model mirrors stack-based execution through nested Environment objects, though without explicit stack frames.

First-class functions are implemented via the Procedure class wrapping, demonstrating higher-order function principles. Error handling for arithmetic operations incorporates concepts from exception handling lectures, with overflow checks using 32-bit integer bounds. The `mapcar` implementation utilizes list comprehension and dynamic function application, reflecting functional programming paradigms discussed in class.

5 IMPLEMENTATION DETAILS

The interpreter employs several notable design choices. Boolean values `T` and `NIL` are mapped to Python's native `True` and `False`, with special formatting during output conversion. The output file mechanism uses Python's context manager pattern, ensuring proper file closure even during error conditions.

Environment lookups use chain-of-responsibility pattern through the `find()` method rather than prototype inheritance. Arithmetic operators are wrapped in safety functions (`safe_add`, etc.) to enforce 32-bit integer constraints while maintaining Python's native number types. The `cons` implementation automatically flattens improper lists to match Lisp's expected behavior.

6 CONCLUSION

This project provided significant insights into both Lisp semantics and interpreter design. Implementing special forms like `quote` and `if` deepened understanding of evaluation strategies, while environment management illuminated scoping concepts. Challenges in balancing Python's dynamic typing with Lisp's type expectations highlighted the importance of careful edge case handling.

The experience reinforced the elegance of Lisp's minimal syntax paired with powerful abstraction capabilities. Future improvements

could add proper tail recursion and expand the type system, but the current implementation successfully demonstrates core interpreter concepts while adhering to project requirements.

REFERENCES

- [1] Python Software Foundation. [n. d.]. Classes. Retrieved April 28, 2025 from <https://docs.python.org/3/tutorial/classes.html>
- [2] Peter Norvig. [n. d.]. (How to Write a (Lisp) Interpreter (in Python)). Retrieved April 28, 2025 from <https://docs.python.org/3/tutorial/interpreter.html>